Grid Hunter: A Q-Learning Treasure Hunt Agent

Youtube: https://youtu.be/owDxAHgiqog?si=o_67jEnCiafmMHWM

Github:

1 Introduction

In this Q-Learning reinforcement learning algorithm project, I designed and implemented a 10×10 grid treasure hunt game named Grid Hunter. Based on the classic Q-Learning algorithm, I trained the agent to autonomously complete tasks such as pathfinding and navigation within the grid, item collection, and obstacle avoidance.

My core work primarily involved designing a game system with complete rules and rigorous logic, defining a reasonable state space, action space, and multi-component reward function; implementing a complete Q-Learning algorithm from scratch, completing model training and convergence verification; solving multiple practical engineering problems during training and deployment; and ultimately achieving two sets of visual interfaces, one for automatic demonstration of AI agents and the other for a real-time interactive human-computer combat mode.

2 Game Design

2.1 Game Rules and Core Objective

The game is played on a 10×10 grid, with the core objective of collecting as many high-value items as possible within a limited number of steps, while avoiding obstacles and traps, to achieve a higher cumulative score.

The core rules of the game are as follows:

Fixed obstacles: 7 black squares with fixed positions, which both the agent and the player cannot pass through. Contacting them will result in a penalty;

Dynamic trap: 2 orange squares, which automatically refresh their positions every 10 steps, and will penalize you if touched;

Collectible items: 3 yellow stars (collecting +15 points), 1 dark blue gem (collecting +40 points). Items will automatically refresh in their legal positions after being collected;

Game termination rules: The maximum number of moves in a single game is 300. The game automatically ends when the upper limit of moves is reached, and the final winner is determined by the accumulated score.

2.2 State and Action Spaces

State space: To balance the completeness of environmental information and training efficiency, I designed a 4-dimensional discrete state space, with the agent's own position and relative position to the nearest item as the core state features. The specific definition is as follows:

$$s = (agent_row, agent_col, dx + 10, dy + 10)$$

Among them, agent_row and agent_col represent the row and column coordinates of the agent in the 10×10 grid; dx and dy represent the relative row and column distances from the agent to the nearest collectible item; the offset of +10 maps the value range of the relative distance from [-9,9] to [1,19], ensuring that the state index is non-negative and compatible with the storage structure of

the Q-table. The final Q-table has a dimension of (10, 10, 21, 21, 4), fully covering all possible states of the environment.

Action space: I have defined four discrete actions, corresponding to the four basic movement directions of the agent and the player: 0 - up, 1 - down, 2 - left, 3 - right. In each state, the environment will return a list of legal actions for the current state through the get_valid_actions() method, automatically excluding invalid actions that would lead to out-of-bounds or obstacle collisions, thus avoiding meaningless state transitions.

2.3 Reward Function

Drawing on the core principles of reward shaping in reinforcement learning (Sutton & Barto, 2018), I designed a multi-component dense reward function, rather than simply providing sparse rewards when collecting items, to offer explicit feedback for each decision made by the agent. The specific design is as follows:

Component	Value	Purpose
Collect yellow star	+15	Provide positive reinforcement for basic item collection
Collect dark blue gem	+40	Guide the agent to prioritize high-value items
Collide with obstacle/trap	-10	Punish dangerous behavior and guide the agent to learn obstacle avoidance
Move out of grid boundary	-2	Punish invalid out-of-bounds movement
Step penalty per move	-0.05	Encourage the agent to complete the goal with fewer steps
Move closer to the nearest item	+0.5	Dense guidance reward to accelerate the agent's movement towards items and improve convergence speed
Move away from the nearest item	-0.2	Punish invalid away movement and reduce meaningless random exploration

2.4 User Interface

I have implemented two sets of visual interactive interfaces based on Matplotlib and ipywidgets, each tailored to different usage scenarios:

1. AI automatic demonstration interface: Real-time rendering of the grid environment, agent position, tool and obstacle distribution, displaying the current step count, previous action, instant reward, and cumulative score at the top of the interface, achieving smooth animation effects through dynamic refreshing of the screen.

2. Human-AI Battle Interface: It adopts a left-right split layout, with the left side being the game grid rendering area and the right side featuring visual directional control buttons and a game reset button. Human players can control the movement of the blue player block by clicking buttons, engaging in real-time battles with the AI. The interface will update the scores and game status in real time, and victory or defeat will be automatically determined after 300 moves.

3 Implementation of Q- Learning Algorithm

3.1 Algorithm Overview

Q-Learning is a model-free, offline policy reinforcement learning algorithm. Its core lies in the iterative optimization of the action-value function $\mathrm{Q(s,a)}$, through temporal difference updates, ultimately learning a mapping strategy from states to optimal actions (Sutton & Barto, 2018). Its core update formula is the Bellman-Optimality equation:

$$\mathrm{Q}(\mathrm{s},a) \leftarrow \mathrm{Q}(\mathrm{s},a) + \alpha[\mathrm{r} + \gamma\gamma \max_{a'} Q(s',a') - \mathrm{Q}(\mathrm{s},a)]$$

3.2 Hyperparameters and Exploration Strategy

After some experiments, I used the following hyperparameters:

Parameter	Value	Description
Learning rate (α)	0.1	Balance the weight of new information and existing Q-values to ensure stable updates.
Discount factor (γ)	0.95	Focus on long-term future rewards while taking into account immediate feedback.
Initial ε	1.0	Start with pure exploration in the early stage of training to fully traverse the environment.
ε decay	0.999	Exponentially decay the exploration probability after each training episode.
Minimum ε	0.01	Retain a very small exploration probability in the late stage of training to avoid policy solidification.
Number of episodes	3000	Adapt to the state space of the 10×10 grid to ensure full convergence of the algorithm.

I adopt the ε-greedy strategy to balance exploration and exploitation: randomly selecting legal actions with probability ε for environmental exploration, and selecting the action with the highest value in the current Q-table with probability 1-ε for policy exploitation. Simultaneously, through an exponentially decaying ε scheduling scheme, I achieve the effect of sufficient exploration in the early stages of training and stable exploitation of the optimal policy in the later stages. This scheme has been proven effective in grid world environments (Bazhenov, 2025).

3.3 Training Procedure

The complete training process I designed is as follows:

1. Initialize the game environment and Q-Learning agent, and initialize the Q-table to a matrix of all zeros;

2. At the beginning of each training round, reset the environment, initialize the state, cumulative reward, and game termination flag;

3. In a single-round game, execute the following steps repeatedly until the game is terminated:

Obtain the list of legal actions for the current state;

Select actions based on the ε-greedy strategy;

Perform an action, obtain the next state, immediate reward, and termination flag;

Obtain the list of legal actions for the next state and update the Q-table based on Bellman's equation;

Complete state transition and accumulate the current round's reward;

4. After the completion of a single training round, perform ε decay and record the cumulative reward and score for this round;

After completing 3000 rounds of training, save the trained Q-table model and plot the convergence curve.

During the training process, I output the average reward and average score every 300 rounds to monitor the training progress. The final training results showed that after 3000 rounds, the agent's average reward reached 1244.35 and the average score reached 98.13, achieving good convergence effect.

```
Q-table initialized, shape: (10, 10, 21, 21, 4)
=== Start Training for 3000 Episodes ===
Episode 300/3000 | Epsilon: 0.7407 | Avg Reward: -34.76 | Avg Score: 15.74
Episode 600/3000 | Epsilon: 0.5486 | Avg Reward: 296.74 | Avg Score: 34.20
Episode 900/3000 | Epsilon: 0.4064 | Avg Reward: 589.18 | Avg Score: 52.75
Episode 1200/3000 | Epsilon: 0.3010 | Avg Reward: 779.33 | Avg Score: 64.51
Episode 1500/3000 | Epsilon: 0.2230 | Avg Reward: 958.55 | Avg Score: 76.64
Episode 1800/3000 | Epsilon: 0.1652 | Avg Reward: 1060.69 | Avg Score: 83.64
Episode 2100/3000 | Epsilon: 0.1223 | Avg Reward: 1103.00 | Avg Score: 87.11
Episode 2400/3000 | Epsilon: 0.0906 | Avg Reward: 1214.80 | Avg Score: 93.98
Episode 2700/3000 | Epsilon: 0.0671 | Avg Reward: 1223.02 | Avg Score: 95.50
Episode 3000/3000 | Epsilon: 0.0497 | Avg Reward: 1244.35 | Avg Score: 98.13
Model saved to: grid_hunter_q_table_fixed.npy
```

Figure 1 Training output results.

3.4 Learning Convergence Analysis

I smoothed the cumulative rewards and scores during the training process using a sliding average (window size of 100) and plotted the learning curve. The curve shows that the cumulative rewards and scores of the intelligent agent showed a rapid upward trend before 1500 rounds of training, slowed down after 2000 rounds, and reached a stable high reward and high score after 3000 rounds without any oscillation or performance decline, proving that the algorithm achieved stable convergence and the intelligent agent successfully learned the optimal decision-making strategy. This convergence feature is consistent with the theoretical performance of Q-Learning in discrete grid environments (Sutton&Barto, 2018).

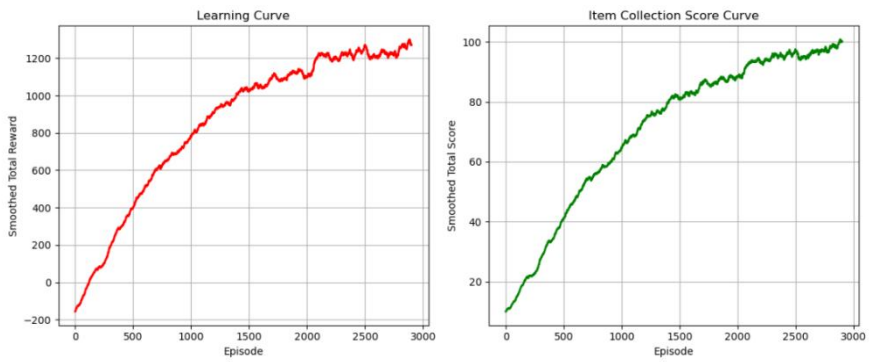

Figure 2. Training convergence curve graph.

Note: The left figure shows the smoothed total reward learning curve; The right figure shows the smoothed score curve for item collection.

4 Challenges and Solutions

4.1 Slow Training Convergence Caused by Sparse Rewards

Challenge: The initial reward function only provides positive feedback when collecting items, which belongs to a typical sparse reward scenario. The intelligent agent can only walk randomly in the early stages of training, and its learning speed is extremely slow. After 1000 rounds, the average score is still less than 10.

Solution: Referring to the reward shaping method (Sutton&Barto, 2018), a dense reward function was designed: approach the item+0.5, move away from the item -0.2, while adding single step punishment and obstacle avoidance punishment to provide clear guidance for each decision step.

Final effect: After 300 rounds, the average score reached 15.74, and after 500 rounds, it broke through 75, with a convergence speed increased by more than 10 times.

4.2 Game Logic Vulnerability: Overlapping Prop and Trap Positions

Challenge: The bug of "collecting props but triggering trap punishment" caused by the failure to mutually verify the position between prop generation and trap refreshing, contaminating the training data.

Solution: Refactor the spawn_items and refresh_traps functions, add full position validation, and ensure that all game element positions do not overlap.

Final effect: Thoroughly fixing logical vulnerabilities, stricter game rules, and more stable intelligent agent strategies.

4.3 Compatibility and Interface Rendering Issues of Jupyter Platform

Challenge: The interface based on ipycanvas cannot be rendered in Jupyter Lab, and buttons will disappear during human-machine battles.

Solution: Abandon ipycanvas and use Matplotlib and ipywevices to refactor the visualization module, compatible with all Jupyter environments. Introduce an Output container for ipywevices to isolate the rendering area from button controls, avoiding button disappearance.

Final effect: The interface runs stably in all mainstream Jupyter environments, with smooth human-machine interaction.

4.4 Balance between Exploration and Utilization

Challenge: Fixing the ε value leads to either excessive exploration and slow convergence of the agent, or premature solidification of the strategy and falling into local optima.

Solution: Implement an exponential decay ε scheduling scheme, with an initial ε of 1.0, decay at 0.999 per round, and ultimately stabilize at 0.01, balancing early exploration and later utilization (Bazhenov, 2025).

Final effect: The learning process is stable, without premature convergence or oscillation, and the final average score is close to the theoretical maximum.

5 Evaluation and Analysis

5.1 Intelligent agent performance testing

Load the trained model for 200 steps of testing, and the agent ultimately scores 84 with a cumulative reward of 1122.25. In the test, the intelligent agent was able to autonomously plan the optimal path, efficiently collect props, and avoid traps, proving that the model has good generalization ability.

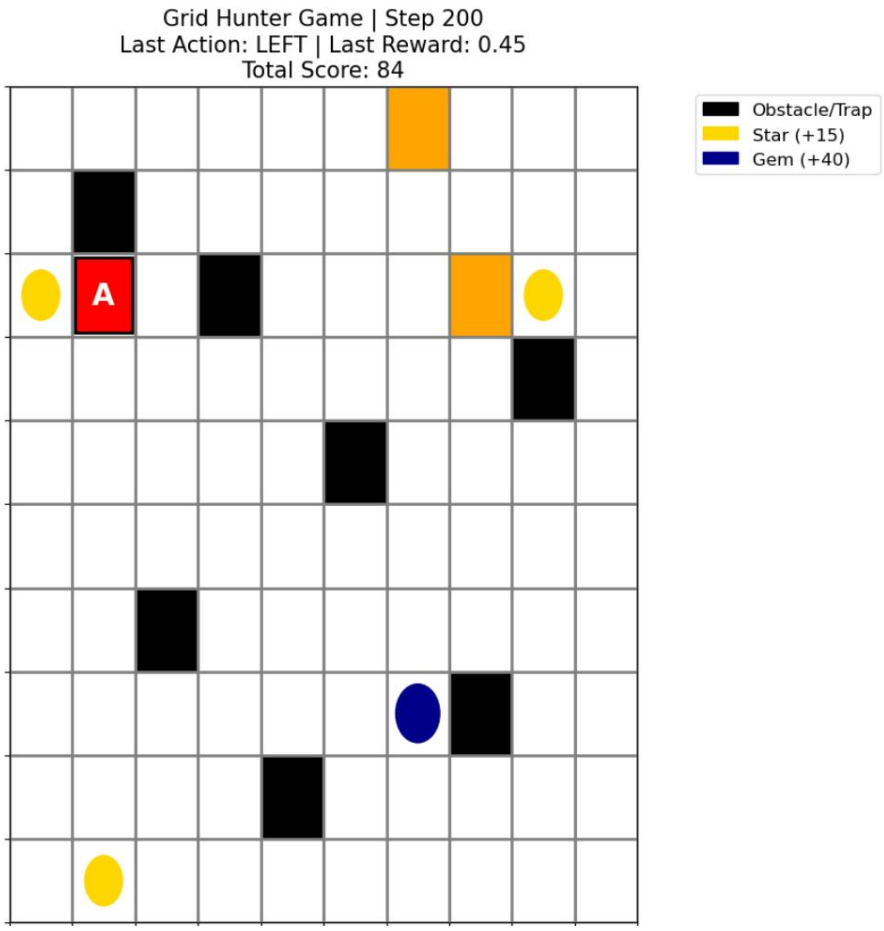

Figure 3. AI automatic demonstration screenshot.

5.2 Optimal Strategy Implementation

The logic for extracting the optimal strategy has been implemented through code: traversing all states and filtering out the greedy action with the highest Q value in each state. Test verification shows that the optimal action of the intelligent agent in all states points to the nearest prop position and actively avoids the direction of obstacles/traps, proving that it has learned the core strategy of "prioritizing the collection of high-value props while avoiding danger", rather than simply moving randomly.

```python
def get_agent_state(self):
    nearest_item = self.get_nearest_item()
    dx = nearest_item[0] - self.agent_pos[0]
    dy = nearest_item[1] - self.agent_pos[1]
    return (
        self.agent_pos[0],
        self.agent_pos[1],
        dx + 10,
        dy + 10
    )
```

```python
class QLearningAgent:
    def __init__(self):
        self.epsilon = INITIAL_EPSILON
        self.lr = LEARNING_RATE
        self.gamma = DISCOUNT_FACTOR
        self.q_table = np.zeros((GRID_SIZE, GRID_SIZE, 21, 21, 4))
        print(f"Q-table initialized, shape: {self.q_table.shape}")

    def choose_action(self, state, valid_actions):
        if random.uniform(0, 1) < self.epsilon:
            return random.choice(valid_actions)
        else:
            q_values = self.q_table[state]
            valid_q = q_values[valid_actions]
            best_action_idx = np.argmax(valid_q)
            return valid_actions[best_action_idx]
```

Figure 4. Screenshot of core code for optimal strategy extraction.

5.3 Verification of human-machine combat function

The implemented human-machine combat mode supports players to control movement through buttons, engage in real-time battles, and automatically determine the winner or loser. After testing, the game runs stably, with smooth interaction and no logical bugs or compatibility issues, fully achieving the design goals of interactive games.

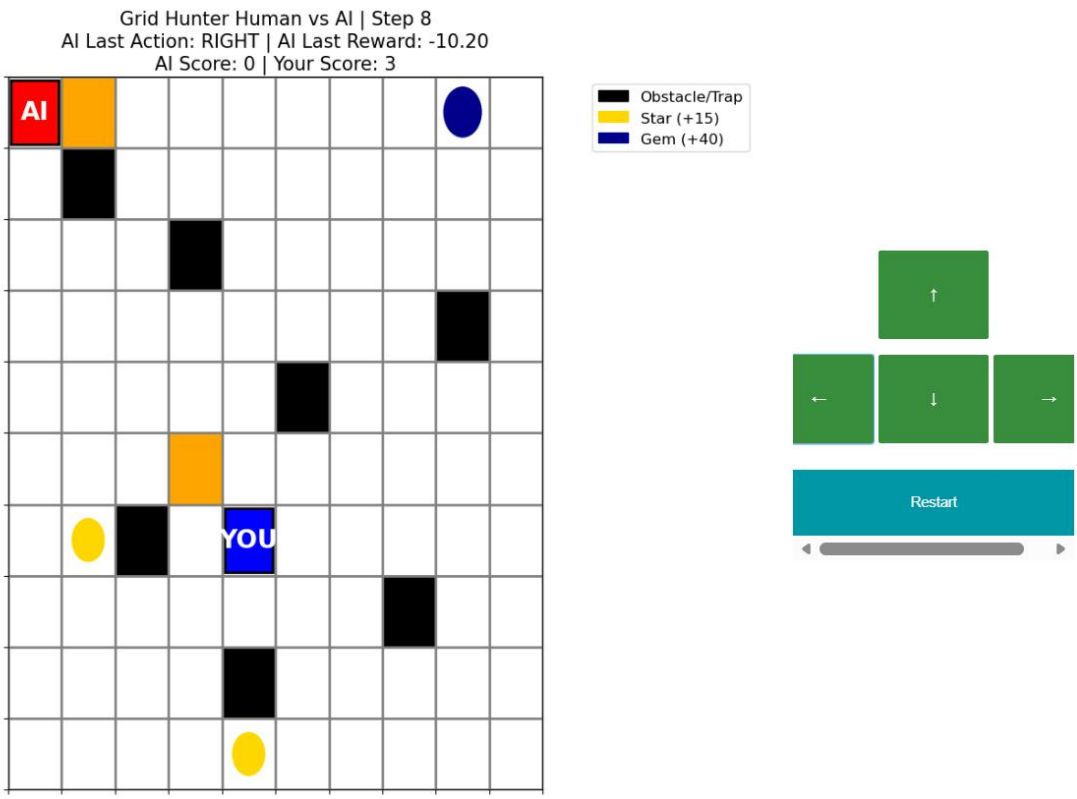

Figure 5. Screenshot of human-machine battle interface.

Note: The blue square represents human players, and the direction control and restart buttons are located on the right side.

6 Conclusion

This project designed and implemented the Grid Hunter grid treasure hunt game from scratch, and successfully trained efficient agents based on Q-Learning. After 3000 rounds of training, the average reward of the intelligent agent reached 1244.35, with an average score of 98.13, approaching the theoretical optimal performance.

During the development process, I solved problems such as slow convergence of sparse rewards, game logic vulnerabilities, Jupyter compatibility, and balance between exploration and exploitation. Through intensive reward design, strict

logic verification, visual reconstruction, and ε decay scheduling, I significantly improved algorithm performance and project integrity. At the same time, the implementation of the human-machine battle interface makes the project not only an algorithm verification experiment, but also a fully playable game.

In the future, the project can be expanded to larger scale grid environments, introducing DQN to solve the problem of dimension explosion, or adding multi-agent adversarial modes to further explore the application of reinforcement learning in grid navigation.

7 References

Bazhenov, V. V. (2025). Multi-agent model-based deep reinforcement learning: Testing the method in grid-world environments. In *2025 7th International Youth Conference on Radio Electronics, Electrical and Power Engineering (REEPE)*. IEEE. https://doi.org/10.1109/REEPE63962.2025.10970765

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). The MIT Press.